

2-Month Work Schedule

Autonomous Research Assistant — Capstone Master Project

Duration: 8 Weeks ($\approx$ 60 working hours total — $\sim$ 7-8 hrs/week) **Stack:** Python · LangChain · Claude API · ChromaDB · Streamlit · Plotly **Goal:** Working agentic research assistant with full dashboard by end of Week 8

Project Phases at a Glance

Phase	Weeks	Focus	Deliverable
1	1-2	Foundation & Setup	Working ReAct loop with 1 tool
2	3-4	Data Pipeline & RAG	Multi-source fetch + ChromaDB retrieval
3	5-6	Agent Intelligence	Self-critique, memory, query decomposition
4	7-8	Dashboard & Evaluation	Full Streamlit UI + final report

Pre-Start Checklist (Do before Week 1)

- ☐ Create a GitHub repo: `research-assistant-capstone`
- ☐ Set up Python virtual environment (`python -m venv venv`)
- ☐ Get API keys: **Anthropic** (Claude), **SerpAPI**, **Semantic Scholar** (free)
- ☐ Install base packages: `pip install anthropic langchain chromadb streamlit plotly spacy`
- ☐ Download spaCy model: `python -m spacy download en_core_web_lg`
- ☐ Register on ArXiv (no key needed) and test PubMed Entrez API
- ☐ Set up a `.env` file for all keys — never hardcode them
- ☐ Create folder structure (see below)

Recommended Folder Structure

```
research-assistant/  
├── agent/  
│   └── __init__.py
```

```

|   |─ react_loop.py      # Core ReAct controller
|   |─ tools.py           # All tool definitions
|   |─ memory.py          # Short + long-term memory
|   |─ critic.py          # Self-critique module
|─ data/
|   |─ ingest.py          # Fetch + chunk + embed
|   |─ sources/           # ArXiv, PubMed, SerpAPI connectors
|   |─ vectorstore.py     # ChromaDB wrapper
|─ nlp/
|   |─ decompose.py       # Query decomposition with spaCy
|   |─ embeddings.py      # BAAI/bge wrapper
|─ dashboard/
|   |─ app.py             # Main Streamlit app
|   |─ charts.py          # All Plotly chart functions
|─ tests/
|─ .env
|─ requirements.txt
|─ README.md

```

PHASE 1 — Foundation & Setup (Weeks 1-2)

Week 1 — Environment, APIs & Basic Agent

Goal: Have a simple agent that can call one tool and return a result.

Day 1-2: Project Setup

- ☐ Initialize GitHub repo, set up `.gitignore` (exclude `.env`, `venv/`)
- ☐ Create `requirements.txt` with pinned versions
- ☐ Write a basic `hello_claude.py` — call Claude API, confirm response
- ☐ Test Anthropic SDK: `client = anthropic.Anthropic()` → `client.messages.create(...)`
- ☐ Confirm SerpAPI returns results for a test query

⚠ Common issue: Anthropic SDK version mismatches. Pin to `anthropic>=0.30.0`.

Day 3-4: First Tool + ReAct Skeleton

- ☐ Build `tools.py` — implement `web_search(query)` using SerpAPI
- ☐ Write `react_loop.py` — implement the 3-step loop:

```
THINK: What do I need to do?
ACT:   Call a tool
OBS:   Read the result, update context
```

- ☐ Hardcode one test question: *"What is RAG in AI?"*
- ☐ Confirm the agent reasons, calls search, reads result

Day 5: Prompt Engineering

- ☐ Write the system prompt for Claude Opus 4.5 — include:
 - Role definition ("You are a research assistant agent...")
 - Tool descriptions and when to use each
 - Output format instructions (JSON tool calls)
 - ☐ Test 3 different research questions; inspect reasoning quality
 - ☐ Log all agent steps to console for debugging
 - ☒ **Week 1 Milestone:** Agent receives a question, calls web search, returns a 1-paragraph answer.
-

Week 2 — Multi-Tool Agent + ArXiv Connector

Goal: Agent can use multiple tools and fetch real academic papers.

Day 1-2: ArXiv & PubMed Connectors

- ☐ Build `sources/arxiv.py`:
 - Use `arxiv` Python package (`pip install arxiv`)
 - Function: `fetch_arxiv(query, max_results=5)` → returns list of `{title, abstract, url, authors}`
- ☐ Build `sources/pubmed.py`:
 - Use `Bio.Entrez` from Biopython (`pip install biopython`)
 - Function: `fetch_pubmed(query, max_results=5)` → same schema
- ☐ Test both with topic: *"transformer attention mechanisms"*

Day 3: Tool Registry

- ☐ Register all tools in LangChain's `Tool` class with name, description, func
- ☐ Give Claude clear descriptions of each tool — this is critical for correct tool selection
- ☐ Add a `tool_dispatcher(tool_name, tool_input)` router function

Day 4-5: Error Handling & Logging

- ☐ Wrap every API call in `try/except` — APIs fail; the agent must recover gracefully
- ☐ Add retry logic (3 attempts with exponential backoff)
- ☐ Set up Python `logging` to write agent steps to `logs/agent.log`
- ☐ Test failure scenarios: bad query, empty results, timeout

✅ **Week 2 Milestone:** Agent fetches papers from ArXiv + PubMed, selects the most relevant 3, and summarizes them in a paragraph.

PHASE 2 — Data Pipeline & RAG (Weeks 3-4)

Week 3 — PDF Parsing & Vector Store

Goal: Agent can ingest a PDF, chunk it, embed it, and retrieve relevant passages.

Day 1-2: PDF Parsing

- ☐ Install: `pip install pymupdf pdfplumber`
- ☐ Build `data/ingest.py`:
 - `parse_pdf(file_path)` → extract raw text using PyMuPDF
 - `chunk_text(text, chunk_size=512, overlap=50)` → use LangChain's `RecursiveCharacterTextSplitter`
 - Each chunk stores: `{text, source, page_num, doc_title}`
- ☐ Test with a real AI paper PDF (e.g., "Attention Is All You Need")
- ☐ Print chunk count and first 3 chunks to verify

Day 3: Embeddings

- ☐ Install: `pip install sentence-transformers`
- ☐ Build `nlp/embeddings.py`:
 - Load `BAAI/bge-small-en-v1.5` from HuggingFace
 - Function: `embed(texts: list[str])` → returns list of float vectors
- ☐ Test: embed 10 chunks, check vector dimensions (should be 384)
- ☐ Compare cosine similarity between a query and chunks manually

Day 4-5: ChromaDB Setup

- ☐ Install: `pip install chromadb`
- ☐ Build `data/vectorstore.py`:
 - `init_collection(name)` → create/load ChromaDB collection

- `upsert_chunks(chunks, embeddings)` → store with metadata
 - `retrieve(query, top_k=5)` → embed query, return top-K chunks
- ☐ End-to-end test: PDF → parse → chunk → embed → store → retrieve
-  **Note:** Use persistent ChromaDB (`chromadb.PersistentClient`) so data survives restarts.

☒ **Week 3 Milestone:** Upload a PDF, retrieve the 5 most relevant chunks for any query.

Week 4 — Full RAG Pipeline + Multi-Source Integration

Goal: Agent pulls from ALL sources, ranks results, and uses RAG for context.

Day 1-2: Semantic Scholar Connector

- ☐ Build `sources/semantic_scholar.py`:
 - Use S2 REST API (free, no key needed for basic use)
 - `fetch_s2(query, limit=5)` → returns papers with citation count, abstract, year
- ☐ Add relevance scoring: combine BM25 keyword score + embedding cosine score
- ☐ Install `rank_bm25` package: `pip install rank-bm25`

Day 3: Query Decomposition with spaCy

- ☐ Build `nlp/decompose.py`:
 - Load `en_core_web_lg`
 - Extract named entities (NER) from the user query
 - Extract keyphrases using `KeyBERT` (`pip install keybert`)
 - Output: list of sub-queries to dispatch in parallel
- ☐ Example: "What are recent advances in RAG for medical AI?" → Sub-queries: `["RAG retrieval augmented generation", "medical AI NLP 2024", "RAG clinical applications"]`

Day 4-5: Combine Everything into the Agent

- ☐ Update `react_loop.py` — agent now has 4 tools:
 1. `web_search` (SerpAPI)
 2. `fetch_papers` (ArXiv + PubMed + S2 combined)
 3. `retrieve_from_db` (ChromaDB RAG)
 4. `summarize_doc` (BART or Claude Sonnet 4.5)
- ☐ Test end-to-end: question → decompose → fetch → store → retrieve → answer
- ☐ Measure: does RAG give better answers than web search alone?

✓ **Week 4 Milestone:** Full data pipeline working. Agent fetches from 4 sources, stores in ChromaDB, retrieves relevant chunks for any research question.

📅 PHASE 3 — Agent Intelligence (Weeks 5–6)

Week 5 — Self-Critique & Memory

Goal: Agent improves its own output and remembers past sessions.

Day 1-2: Self-Critique Module

- ☐ Build `agent/critic.py`:
 - Use **Claude Haiku 4.5** (fast, cheap) as the critic
 - System prompt: *"You are a research quality reviewer. Evaluate the following report on: completeness, factual consistency, citation coverage, and clarity. Return a JSON with scores and specific revision requests."*
 - Function: `critique(report_text, original_query)` → `{scores, issues, suggestions}`
- ☐ Integrate into `react_loop.py`: after first draft, run critic → if score < threshold → revise
- ☐ Set max 2 revision loops to avoid infinite iteration

Day 3: Short-Term Memory

- ☐ Add conversation history to the agent context:

```
python

history = [] # list of {role, content} dicts
# Pass history into every Claude API call
```

- ☐ Test: ask a follow-up question — agent should remember the first question
- ☐ Limit history to last 10 exchanges to avoid context overflow

Day 4-5: Long-Term Memory (Session Persistence)

- ☐ Use SQLite to store past queries + reports:

```
python

# schema: sessions(id, query, report, sources, timestamp, score)
```

- ☐ Build `agent/memory.py`:
 - `save_session(query, report, sources, score)`

- `get_similar_sessions(query, top_k=3)` — embed query, find similar past sessions

☐ Surface past sessions in the Streamlit sidebar (Week 7)

☒ **Week 5 Milestone:** Agent self-critiques and revises its report. Past research sessions are saved and retrievable.

Week 6 — End-to-End Testing & Refinement

Goal: The full pipeline works reliably on diverse topics.

Day 1-2: Structured Report Generation

☐ Define report schema (use Claude Opus 4.5 with structured output):

```
## Executive Summary      (2-3 sentences)
## Key Findings            (3-5 bullet points)
## Detailed Analysis      (2-3 paragraphs)
## Identified Gaps        (what research is missing)
## References              (auto-generated citations)
```

- ☐ Use Claude's JSON mode to enforce structure
- ☐ Generate citation strings in APA format automatically

Day 3: Quality Testing

☐ Run 5 diverse test queries across different domains:

1. *"Transformer architectures in NLP"*
2. *"RAG for medical question answering"*
3. *"Reinforcement learning from human feedback"*
4. *"LLM hallucination mitigation techniques"*
5. *"Vector databases comparison: Pinecone vs ChromaDB"*

☐ Log: retrieval count, agent steps, revision count, final score, latency

Day 4-5: Debugging & Optimization

- ☐ Fix top bugs found in testing
- ☐ Reduce latency: run API calls in parallel using `asyncio`
- ☐ Add rate limiting protection for SerpAPI (100 calls/month on free tier)
- ☐ Write unit tests for: chunking, embedding, retrieval, critique

☒ **Week 6 Milestone:** End-to-end pipeline passes 5/5 test queries. Average report quality score $\geq 7/10$ from the critic.

PHASE 4 — Dashboard & Evaluation (Weeks 7-8)

Week 7 — Streamlit Dashboard

Goal: Full interactive UI wrapping the agent backend.

Day 1-2: App Shell & Chat Interface

- ☐ Build `dashboard/app.py`:

```
python

import streamlit as st
st.set_page_config(page_title="Research Assistant", layout="wide")
```

- ☐ Build sidebar: New Query | History | Settings
- ☐ Add `st.chat_input()` for query entry
- ☐ Add `st.chat_message()` for agent responses
- ☐ Add `st.progress()` + `st.status()` for live step tracking

Day 3: Results Display

- ☐ Tab 1 — **Report:** render markdown with `st.markdown()`
- ☐ Tab 2 — **Sources:** expandable cards for each paper (title, abstract, link, score)
- ☐ Tab 3 — **Charts:** Plotly visualizations (see Day 4)
- ☐ Export buttons: download as `.md`, `.pdf`, `.txt`

Day 4: Plotly Charts (`dashboard/charts.py`)

- ☐ `relevance_bar_chart(sources_df)` — `px.bar` with source name vs score
- ☐ `topic_scatter(embeddings_2d, labels)` — `px.scatter` using UMAP-reduced embeddings
- ☐ `agent_timeline(steps_log)` — `px.timeline` showing each agent action + duration
- ☐ `confidence_gauge(score)` — `go.Indicator(mode='gauge+number')`
- ☐ Install UMAP: `pip install umap-learn`

Day 5: Session History & Settings

- ☐ Load past sessions from SQLite into sidebar
- ☐ Click a past session → reload that report instantly
- ☐ Settings panel: max sources, model choice (Opus/Sonnet), revision rounds toggle

✓ **Week 7 Milestone:** Full Streamlit app running locally. Enter query → see live progress → get report with charts.

Week 8 — Evaluation, Polish & Submission

Goal: Evaluate the system, write the report, prepare the presentation.

Day 1: Quantitative Evaluation

- ☐ Precision@5: for 5 test queries, manually label which retrieved chunks are relevant → compute score
- ☐ ROUGE-L: compare agent summaries to manually written gold summaries
- ☐ Latency: measure time per stage (fetch, embed, retrieve, generate, critique)
- ☐ Compile results into a table

Day 2: User Testing

- ☐ Ask 2-3 peers to use the app with their own research questions
- ☐ Collect feedback: Was the report useful? Were sources relevant? Was the UI clear?
- ☐ Fix top 3 usability issues

Day 3: Code Polish

- ☐ Write `README.md` with setup instructions and screenshots
- ☐ Clean up all `print` statements → replace with proper logging
- ☐ Add docstrings to all functions
- ☐ Final `requirements.txt` update: `pip freeze > requirements.txt`
- ☐ Push clean final version to GitHub

Day 4: Final Report / Paper

- ☐ Write project report covering:
 - Problem statement & motivation
 - System architecture diagram
 - Data pipeline description
 - Model choices & justification
 - Evaluation results
 - Limitations & future work
- ☐ Aim for 8-12 pages including figures

Day 5: Presentation Prep

- ☐ Finalize the PowerPoint slides (already built!)
- ☐ Prepare a live demo: 2 research queries, walk through UI
- ☐ Practice presenting: keep demo under 5 minutes
- ☐ Prepare for questions on: why RAG, why Claude, how critique works
- ☒ **Week 8 Milestone: PROJECT COMPLETE** ✓ Working app ✓ Evaluation results ✓
Written report ✓ Presentation ready

 Risk Register & Mitigation

Risk	Likelihood	Impact	Mitigation
API rate limits (SerpAPI free tier: 100/mo)	High	Medium	Cache all API results locally in JSON files
Claude API costs exceed budget	Medium	High	Use Haiku 4.5 for testing, Opus only for final runs
ChromaDB memory issues on large corpus	Low	Medium	Limit ingestion to 50 docs per session
spaCy model slow on low-RAM machine	Medium	Low	Fall back to NLTK RAKE if needed
Week 6 testing reveals major architecture flaw	Low	High	Keep Phase 1-2 code modular so components can be swapped
Streamlit deployment issues	Low	Low	Keep app local; use <code>streamlit run app.py</code> for demo

 Estimated API Cost Budget (2 months)

Service	Usage	Est. Cost
Claude Opus 4.5 (final runs only)	~50 calls × 4K tokens	~\$8
Claude Sonnet 4.5 (development)	~200 calls × 2K tokens	~\$6
Claude Haiku 4.5 (critique, testing)	~500 calls × 1K tokens	~\$1

Service	Usage	Est. Cost
SerpAPI	Free tier (100 calls/mo)	\$0
ArXiv / PubMed / S2 APIs	Free	\$0
Total		~\$15-20

 Set a **\$25** hard limit on your Anthropic API dashboard under **Usage Limits** to prevent surprises.

 Key Packages & Versions

```
anthropic>=0.30.0
langchain>=0.2.0
langchain-community>=0.2.0
chromadb>=0.5.0
sentence-transformers>=3.0.0
streamlit>=1.36.0
plotly>=5.22.0
pymupdf>=1.24.0
pdfplumber>=0.11.0
spacy>=3.7.0
keybert>=0.8.0
rank-bm25>=0.2.2
biopython>=1.84
arxiv>=2.1.0
umap-learn>=0.5.6
pandas>=2.2.0
sqlalchemy>=2.0.0
python-dotenv>=1.0.0
```

 Weekly Rhythm (Suggested)

Day	Activity
Monday	Review previous week, plan tasks, set up environment for the week
Tue-Thu	Core coding sessions (2-3 hrs each)

Day	Activity
Friday	Testing, debugging, commit to GitHub, write brief progress note
Weekend	Light reading / review documentation (optional)

Built on NLP + Gen AI coursework · Designed for an 8-week summer timeline · Claude API (Anthropic)